

파이썬을 이용한 알기쉬운 수치해석

김시조

국립경국대학교: (구)국립안동대학교

February 17, 2026

머릿말

Note. 참고 URL

<https://github.com/seejokim/na-python>

21세기는 기술 혁신과 지식의 융합이 핵심 동력으로 작용하는 시대다. 특히 4차 산업혁명과 인공지능 기술의 발전은 전통적인 공학 분야에 새로운 도전과 기회를 제공하고 있다. 이러한 환경에서 수치해석은 공학 문제를 해결하는 핵심 도구로, 다양한 데이터 기반 분석과 최적화 문제 해결에 활용되고 있다.

현재 수치해석의 중요성은 더욱 강조되고 있으며, 특히 자율주행 자동차의 수치해석에서 가장 중요한 것은 정확도와 실시간 처리이다. 자율주행 차량은 다양한 센서 데이터를 실시간으로 처리하며, 도로 상황을 정확하게 예측하고 반응해야 하기 때문이다. 이러한 기술은 수치해석을 통해 더욱 정교해지고, 안전한 자율주행을 가능하게 한다.

또한 AI와 수치해석은 깊은 관계를 맺고 있으며, AI의 발전이 수치해석 방법론에 많은 영향을 미쳤고, 반대로 수치해석 기술이 AI의 성능을 향상시키는 데 중요한 역할을 해왔다. 수치해석은 실세계의 문제를 수학적으로 모델링하고 해결하기 위한 다양한 알고리즘과 방법을 제공하며, AI는 이러한 수학적 모델을 데이터 기반으로 학습하고 최적화하는 능력을 갖추고 있다. 이 둘의 결합은 현대 과학과 공학의 중요한 연구 분야로 자리 잡았다.

예를 들어, 2024년 노벨 물리학상은 머신러닝과 AI 기술이 물리학 문제 해결에 중요한 기여를 한 연구자에게 수여되었다. 이는 머신러닝이 복잡한 물리적 시스템을 시뮬레이션하거나 예측하는 데 사용되고 있음을 보여준다. 또한 알파고에 이어 알파폴드와 같은 AI 기술이 노벨 화학상을 수상하며, 수치해석 기법을 바탕으로 AI 모델이 실세계 문제를 해결하는 데 어떻게 활용될 수 있는지를 보여주었다.

이 책은 파이썬 프로그래밍 언어를 활용하여 수치해석의 기초부터 심화 내용까지 단계적으로 학습할 수 있도록 구성되었다. 첫 장에서는 파이썬의 기본 문법과 데이터 처리에 필수적인 라이브러리(예: NumPy, Matplotlib)를 다루며, 프로그래밍과 수치해석의 연결고리를 제공한다. 이어지는 장에서는 비선형 방정식 근사법, 선형 연립방정식 해법, 수치미분 및 수치적분, 보간법, 수치 회귀 등 전통적인 수치해석의 핵심 주제들을 체계적으로 소개한다.

특히 후반부에서는 상미분 방정식과 편미분 방정식의 수치적 해법을 다루며, 유한차분법(Finite Difference Method)과 유한요소법(Finite Element Method)의 응용을 통해 실질적인 문제 해결 방법을 제시한다. 이와 더불어 Python을 활용하여 다양한 수치해석 문제를 직접 해결해보는 연습문제를 포함해 독자가 이론과 실습을 병행하며 학습할 수 있도록 하였다.

김시조

Contents

1	파이썬의 이해	1
2	비선형 방정식 구하기	2
2.1	비선형 방정식 구하기 개요	2
2.2	점진탐색법 (Incremental Search Method)	2
2.2.1	알고리즘의 원리	2
2.2.2	알고리즘	2
2.2.3	파이썬 소스 코드 함수 구현	3
2.2.4	예제	3
2.2.5	중근를 갖는 경우	4
2.3	이분법 (Bisection Method)	4
2.3.1	알고리즘의 원리	4
2.3.2	이분법 알고리즘	5
2.3.3	Python SciPy 라이브러리를 활용한 이분법	6
2.4	뉴턴-랩슨법 (Newton-Raphson Method)	6
2.4.1	뉴턴-랩슨법의 설명 (테일러 급수 기반)	7
2.4.2	뉴턴-랩슨법의 설명 (그래프 기반)	8
2.4.3	알고리즘	9
2.4.4	파이썬 소스 코드 함수 구현	10
2.4.5	실습 예제	10
2.4.6	Python SciPy 이용한 뉴턴-랩슨	10
2.5	연습문제	11

Chapter 1

파이썬의 이해

Chapter 2

비선형 방정식 구하기

2.1 비선형 방정식 구하기 개요

비선형 방정식의 해를 구하는 다양한 수치적 방법을 소개한다.

2.2 점진탐색법 (Incremental Search Method)

2.2.1 알고리즘의 원리

점진적으로 구간을 이동하면서 함수값의 부호 변화를 탐색한다.

2.2.2 알고리즘

단계별 절차를 기술한다.

점진탐색법 알고리즘

1. Step 1: 초기 구간 설정

연속함수 $f(x)$ 의 구간 $[a, b]$ 에서 $f(a)f(b) < 0$ 이면 연속함수 $f(x)$ 의 구간 $[a, b]$ 에서 근이 존재한다.

분할 간격을 다음과 같이 설정한다.

$$\Delta x = \frac{b - a}{n}$$

2. Step 2: 초기값 초기화

초기값을

$$x_0 = a$$

로 설정한다.

3. Step 3: 탐색 과정

- 새로운 값 $x_{i+1} = x_i + \Delta x$ 를 계산한다.
- $f(x_i)f(x_{i+1}) < 0$ 을 만족하는지 검사한다.

4. Step 4: 근의 구간 확인 및 갱신

- $f(x_i)f(x_{i+1}) < 0$ 이면 근은 구간 $[x_i, x_{i+1}]$ 에 존재한다.
- 새로운 구간을 $[a, b] = [x_i, x_{i+1}]$ 로 설정한다.

5. Step 5: 구간 세분화

새로운 구간 $[a, b]$ 를 다시 n 등분하여

$$\Delta x = \frac{b - a}{n}$$

으로 재설정한다.

6. Step 6: 종료 조건 검사

- Δx 가 충분히 작아질 때까지 반복한다.
- $\Delta x < \varepsilon$ 이면 계산을 종료한다.

7. Step 7: 예외 처리

- $x_{i+1} > b$ 이면 근이 존재하지 않음을 반환한다.
- $f(a)f(b) > 0$ 이면 탐색 실패를 알린다.

2.2.3 파이썬 소스 코드 함수 구현

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter02/section_02_02_incremental_search.py

```
def incremental_search(f, x_start, x_end, step):
    x = x_start
    while x < x_end:
        if f(x) * f(x + step) < 0:
            return x, x + step
        x += step
    return None
```

2.2.4 예제

주어진 함수의 해

$$f(x) = x^3 - 6x^2 + 11x - 6 = 0$$

을 구간 $[0, 4]$ 에서 찾는다.

Python 코드

```
# define function
f = lambda x: x**3 - 6*x**2 + 11*x - 6

# run incremental_search
x_range = incremental_search(f, 0, 4, 0.1)

# output
```

```

if x_range:
    print("interval for root:", x_range)
else:
    print("No root")

```

output

"interval for root: (2.9, 3.0)"

2.2.5 중근를 갖는 경우

해가 존재하지 않는 경우를 분석한다.

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter02/section_02_02_05_incremental_search_multiple_root.py

2.3 이분법 (Bisection Method)

2.3.1 알고리즘의 원리

연속 함수에서 부호 변화 구간을 반씩 줄여가며 해를 찾는다.

중간값 정리 (Intermediate Value Theorem)

함수 $f(x)$ 가 구간 $[a, b]$ 에서 연속이고 $f(a)f(b) < 0$ 이면,

$$\exists c \in (a, b) \quad \text{s.t.} \quad f(c) = 0$$

즉, 구간 $[a, b]$ 안에 적어도 하나의 근이 존재한다.

이분법의 핵심 아이디어

- 중간점 계산

$$c = \frac{a + b}{2}$$

- 부호 검사

$$\begin{cases} f(a)f(c) < 0 \Rightarrow \text{근은 } [a, c] \\ f(c)f(b) < 0 \Rightarrow \text{근은 } [c, b] \end{cases}$$

- 위 과정을 반복하여 구간을 절반씩 줄인다.

```
[scale=1.1]
[->] (-0.5,0) -- (6,0) node[right] x; [->] (0,-2) -- (0,3) node[above] f(x);
[thick,domain=0.5:5,smooth] plot(,0.05*(-1)*(-4)*(2*-7));
(1,0) circle (2pt); (4,0) circle (2pt);
[below] at (1,0) a; [below] at (4,0) b;
[dashed] (2.5,-2) -- (2.5,2); [below] at (2.5,0) c =  $\frac{a+b}{2}$ ;
at (3.5,2) 부호 변화 발생;
```

수렴 특성

$$\text{구간 길이} = \frac{b-a}{2^n}$$

반복 횟수 n 이 증가할수록 오차는 기하급수적으로 감소한다.

요약

이분법은 연속 함수의 부호 변화를 이용하여 구간을 절반씩 줄여가며 근을 찾는 **항상 수렴하는 안정적인 방법**이다.

2.3.2 이분법 알고리즘

이분법 (Bisection Method) 절차

① Step 1 : 초기 조건 설정

- 구간 $[a, b]$ 에서

$$f(a)f(b) < 0$$

을 만족하는지 확인한다.

- 조건이 만족되지 않으면 알고리즘을 종료한다.

② Step 2 : 중간점 계산

- 중간점 계산:

$$c = \frac{a+b}{2}$$

③ Step 3 : 근의 위치 판별

- $f(c)$ 를 계산한다.
- 만약

$$f(a)f(c) < 0$$

이면 근은 $[a, c]$ 구간에 존재하므로

$$b = c$$

로 설정한다.

- 반대로

$$f(c)f(b) < 0$$

이면 근은 $[c, b]$ 구간에 존재하므로

$$a = c$$

로 설정한다.

④ Step 4 : 종료 조건 검사

- 구간 길이

$$|b - a|$$

가 허용 오차 ε 보다 작아지면 알고리즘을 종료한다.

- 이때 c 를 근의 근사값으로 간주한다.

⑤ Step 5 : 반복

- Step 2부터 Step 4까지 과정을 반복한다.
- 원하는 정확도 내에서 근을 계산한다.

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter02/section_02_03_02_bisection_algorithm.py

```
def bisection(f, a, b, tol=1e-6):
    while abs(b - a) > tol:
        c = (a + b) / 2
        if f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2
```

2.3.3 Python SciPy 라이브러리를 활용한 이분법

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter02/section_02_03_03_bisection_scipy.py

```
from scipy.optimize import bisect
root = bisect(lambda x: x**3 - x - 2, 1, 2)
print(root)
```

2.4 뉴턴-랩슨법 (Newton-Raphson Method)

2.4.1 뉴턴-랩슨법의 설명 (테일러 급수 기반)

비선형 방정식

$$f(x) = 0$$

의 해를 구하기 위하여 뉴턴-랩슨(Newton-Raphson) 방법은 함수의 테일러 급수를 이용하여 근을 점진적으로 개선하는 방법이다.

테일러 전개

함수 $f(x)$ 를 x_0 근방에서 전개하면

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 + \dots$$

만약 $x = x_0 + \Delta x$ 라 하면,

$$f(x_0 + \Delta x) = f(x_0) + f'(x_0)\Delta x + \frac{f''(x_0)}{2!}\Delta x^2 + \dots$$

Δx 가 매우 작다고 가정하면 2차 이상 항을 무시할 수 있으므로

$$f(x_0 + \Delta x) \approx f(x_0) + f'(x_0)\Delta x$$

근을 찾기 위해서는

$$f(x_0 + \Delta x) = 0$$

을 만족해야 하므로,

$$0 \approx f(x_0) + f'(x_0)\Delta x$$

이를 Δx 에 대하여 정리하면

$$\Delta x = -\frac{f(x_0)}{f'(x_0)}$$

따라서 새로운 근의 근사값은

$$x_1 = x_0 + \Delta x = x_0 - \frac{f(x_0)}{f'(x_0)}$$

이를 일반화하면 반복 공식은 다음과 같다.

$$\boxed{x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}}$$

수렴 조건

- 초기 추정값 x_0 이 실제 근에 충분히 가까워야 한다.

- 함수 $f(x)$ 는 해당 구간에서 연속이며 미분 가능해야 한다.
- $f'(x_n) \neq 0$ 이어야 한다.

뉴턴-랩슨법은 수렴 속도가 매우 빠른 2차 수렴(quadratic convergence)을 가지지만, 초기값 선택에 따라 발산할 수도 있는 민감한 방법이다.

2.4.2 뉴턴-랩슨법의 설명 (그래프 기반)

뉴턴-랩슨법은 함수의 접선을 이용하여 근을 반복적으로 개선해 나가는 방법이다.

초기 추정값 x_0 에서 함수 $f(x)$ 의 접선을 그리면 그 접선이 x 축과 만나는 점을 새로운 근사값 x_1 로 사용한다.

기하학적 원리

- 초기 추정값: x_0
- 접선의 방정식:

$$y = f(x_0) + f'(x_0)(x - x_0)$$

- $y = 0$ 을 대입하여 다음 근사값 계산:

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

이 과정을 반복하면 일반 반복식은

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

이 된다.

```
[scale=1.1]
[->] (-0.5,0) -- (5.5,0) node[right] x; [->] (0,-2) -- (0,3) node[above] f(x);
[thick,domain=0.5:5,smooth] plot(,0.15*(-1)*(-3)*(-4)+0.5);
(2.8,1.2) circle (2pt); [right] at (2.8,1.2) f(x_0);
[dashed] (1,-1) -- (4.5,2.5);
(2.8,0) circle (2pt); [below] at (2.8,0) x_0;
(2.1,0) circle (2pt); [below] at (2.1,0) x_1;
[->,thick] (2.8,0.3) -- (2.1,0.3);
```

해석

- 곡선은 $y = f(x)$
- 점 $(x_0, f(x_0))$ 에서 접선을 그림
- 접선이 x 축과 만나는 점이 x_1

- 이를 반복하면 근으로 빠르게 수렴

특징

- 2차 수렴 (Quadratic convergence)
- 초기값에 매우 민감
- 도함수 계산 필요

2.4.3 알고리즘

뉴턴-랩슨(Newton-Raphson) 알고리즘

① Step 1 : 초기 추정값 설정

- 함수 $f(x)$ 와 그 도함수 $f'(x)$ 를 정의한다.
- 초기 추정값 x_0 를 설정한다.
- 초기값은 실제 근에 가능한 한 가까운 값으로 선택한다.
- 초기 추정값에서 $f'(x_0) \neq 0$ 이어야 한다.

② Step 2 : 반복 계산

- 현재 추정값 x_n 에서

$$f(x_n), \quad f'(x_n)$$

을 계산한다.

- 다음 근사값을 계산한다.

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

③ Step 3 : 수렴 여부 확인

- 함수값이 허용 오차 범위에 들어왔는지 확인한다.

$$|f(x_{n+1})| < \text{tolerance}$$

- 또는 연속된 두 근사값의 차이를 검사한다.

$$|x_{n+1} - x_n| < \text{tolerance}$$

- 조건을 만족하면 x_{n+1} 을 근의 근사값으로 반환하고 종료한다.
- 만족하지 않으면

$$x_n \leftarrow x_{n+1}$$

로 설정하고 Step 2로 돌아간다.

특징 요약

- 수렴 속도가 매우 빠른 2차 수렴(Quadratic convergence)을 가진다.
- 초기값 선택이 매우 중요하다.
- 도함수 계산이 필요하다.
- $f'(x_n) = 0$ 또는 매우 작은 경우 발산 가능성이 있다.

2.4.4 파이썬 소스 코드 함수 구현

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter02/section_02_04_04_newton_raphson_function.py

```
def newton_method(f, df, x0, tol=1e-6):
    x = x0
    while True:
        x_new = x - f(x)/df(x)
        if abs(x_new - x) < tol:
            return x_new
        x = x_new
```

2.4.5 실습 예제

주어진 함수의 해

$$f(x) = x^3 - 6x^2 + 11x - 6 = 0$$

을 찾는다.

초기값 $x_0 = 2.1$ 로 설정하여 뉴턴-랩슨법을 사용해 구한다.

초기 추정값 $x_0 = 2.1$

반복 1: $x = 2.100000$, $f(x) = -9.900000 \times 10^{-2}$, $f'(x) = -9.700000 \times 10^{-1}$

반복 2: $x = 1.997938$, $f(x) = 2.061847 \times 10^{-3}$, $f'(x) = -9.999872 \times 10^{-1}$

반복 3: $x = 2.000000$, $f(x) = -1.753115 \times 10^{-8}$

뉴턴-랩슨법을 구한 근:

$$x \approx 2.000000175311468$$

2.4.6 Python SciPy 이용한 뉴턴-랩슨

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter02/section_02_04_06_newton_scipy.py

```
from scipy.optimize import newton
root = newton(lambda x: x**3 - x - 2, 1.5)
print(root)
```

2.5 연습문제